Phase 2 - Grade 1 - Linear Algebra

Key Learning Points -- Vectors

A **vector** is an ordered list of numbers describing one 'thing'. Example: a house as [price, size, bedrooms] -> [50, 1200, 3]. ML models only understand numbers, so every 'thing' a model reasons about gets turned into a vector first -- the single most foundational idea in ML.

Analogy that made it click: a vector is like a fixed-size array or simple POJO with numeric fields, except every field must be a number, and ORDER MATTERS -- [50,1200,3] means something different from [1200,50,3] unless the same field order is always kept.

Scalar
: A single, ordinary number with no direction and no multiple components (e.g. 5, -3, 0.01).

Vector addition/subtraction
: Performed ELEMENT-BY-ELEMENT; both vectors must be the same length.

Scalar multiplication
: Multiplying every element of a vector by the same number, stretching/shrinking it uniformly.

Magnitude / Length / Norm
: Three names for the SAME thing: how 'big' a vector is geometrically -- the length of the arrow from the origin to the point it represents. NOT the same as len(vector), which counts elements. Computed via square-root-of-sum-of-squares (an extension of the Pythagorean theorem).

Dot product
: Two vectors of the SAME LENGTH -> ONE scalar: multiply corresponding elements, then sum. Measures how much two vectors point the same direction, combined with size. This is the single most important vector operation in ML -- the exact computation every neuron performs on its inputs and weights.

Formula: Vector Addition / Subtraction

```
v1 + v2 = [v1(1)+v2(1), v1(2)+v2(2), ..., v1(n)+v2(n)]

Example: a=[2,4,6], b=[1,1,1] -> a+b=[3,5,7], a-b=[1,3,5]
```

Formula: Scalar Multiplication

```
k * v = [k*v(1), k*v(2), ..., k*v(n)]

Example: 4 * [2,4,6] = [8,16,24]
```

Formula: Magnitude / Norm

$$||v|| = \sqrt{v(1)^2 + v(2)^2 + \ldots + v(n)^2}$$

Example: $v=[3,4] \rightarrow \sqrt{3^2+4^2} = \sqrt{9+16} = \sqrt{25} = 5$

Example: $v=[6,8] \rightarrow \sqrt{6^2+8^2} = \sqrt{36+64} = \sqrt{100} = 10$

Example: $c=[2,4,6] \rightarrow \sqrt{4+16+36} = \sqrt{56} \approx 7.4833$ (via np.linalg.norm)

Formula: Dot Product

```
a · b = a(1)*b(1) + a(2)*b(2) + ... + a(n)*b(n)

Example: a=[2,4,6], b=[1,0,1] -> (2*1)+(4*0)+(6*1) = 2+0+6 = 8

Example: p=[3,6,9], q=[1,0,-1] -> (3*1)+(6*0)+(9*-1) = 3+0-9 = -6
```

Key Learning Points -- Matrices

A **matrix** is multiple vectors STACKED TOGETHER -- a grid of rows and columns describing MANY things at once. Analogy: exactly a 2-D array (int[][]) or a table with no column names -- the same shape already used with NumPy 2-D arrays and DataFrames in Phase 1.

Matrix dimensions (rows × columns)

Always rows first, then columns -- same convention as NumPy's .shape.

Matrix indexing

matrix[row,col] -> single element. matrix[row,:] -> whole row. matrix[:,col] -> whole column.

Matrix multiplication (A × B)

Every entry in the result is the DOT PRODUCT of one row of A and one column of B. Valid only when A's columns = B's rows ('inner dimensions match') -- because a dot product needs two vectors of equal length, and a row's length is A's column-count while a column's length is B's row-count.

Result size

A (m×n) × B (n×q) -> result (m×q).

Order matters

A×B is generally NOT equal to B×A (unlike plain numbers).

@ vs * in NumPy

@ (or np.matmul) performs TRUE matrix multiplication. * performs ELEMENTWISE multiplication (position-by-position) -- a completely different operation. Using * when @ was intended is a very common mistake.

Formula: Matrix Multiplication Validity & Mechanics

```
A (m × n) × B (n × q) -- valid only if inner numbers (n, n) match. Result is (m × q).

Worked example: A=[[1,2],[3,4]], B=[[5,6],[7,8]]

Result[0,0]=(1*5)+(2*7)=19 Result[0,1]=(1*6)+(2*8)=22

Result[1,0]=(3*5)+(4*7)=43 Result[1,1]=(3*6)+(4*8)=50

A@B = [[19,22],[43,50]]

Worked example: A=[[2,0],[1,3]], B=[[4,1],[2,5]]

Result[0,0]=(2*4)+(0*2)=8 Result[0,1]=(2*1)+(0*5)=2

Result[1,0]=(1*4)+(3*2)=10 Result[1,1]=(1*1)+(3*5)=16

A@B = [[8,2],[10,16]]
```

Clarified Doubts

My doubt: What does 'a.k.a.' mean, and can you explain scalar more?

Clarification: 'a.k.a.' = 'also known as' -- just noting that magnitude/length/norm are three names for the same thing. A scalar is a single ordinary number with no direction (e.g. 5, -3, 0.01), as opposed to a vector, which bundles multiple numbers together. Scalar multiplication 'scales' a vector -- stretches/shrinks every element by the same factor.

My doubt: How can I write power/square-root notation in plain chat?

Clarification: No special formatting needed -- write it as plain text, e.g. sqrt(6^2 + 8^2) = sqrt(36+64) = sqrt(100) = 10, using ^ for powers and sqrt(...) for square roots.

My doubt: Usually 'length' means how many items are in a collection -- why does np.linalg.norm() also get called 'length'?

Clarification: English overloads the word 'length' for two unrelated things: (1) COUNT of elements -- len(c) for c=[2,4,6] gives 3. (2) GEOMETRIC SIZE -- np.linalg.norm(c) for the same c gives ≈7.483. These measure completely different things: picture a vector as an arrow on a graph -- the norm is the physical length of that arrow, unrelated to how many numbers define its coordinates.

Examples

```
import numpy as np

# --- Vectors ---
a = np.array([2, 4, 6]); b = np.array([1, 1, 1])
a + b        # [3 5 7]
a - b        # [1 3 5]
4 * a        # [ 8 16 24]
np.linalg.norm([2,4,6])   # 7.483314773547883
np.dot(np.array([2,4,6]), np.array([1,0,1]))   # 8

# --- Matrices ---
matrix = np.array([[80,90],[70,60],[95,85],[60,70]])   # 4 students x 2 test scores
matrix.shape      # (4, 2)
matrix[2, :]      # [95 85]  -- Student index 2, full row
matrix[:, 1]      # [90 60 85 70]  -- everyone, column index 1
matrix[0, 1]      # 90  -- single value

A = np.array([[2, 0], [1, 3]]); B = np.array([[4, 1], [2, 5]])
A * B          # WRONG for matrix multiply -- elementwise: [[8,0],[2,15]]
A @ B          # CORRECT: [[8, 2], [10, 16]]
```

Quiz Q&A; -- Vectors

Q1. Given p=[3,6,9], q=[1,1,1], compute p+q and 2*p.

A: p+q = [4,7,10]. 2*p = [6,12,18].

Q2. Compute the magnitude of v=[6,8], showing all steps.

A: sqrt(6^2+8^2) = sqrt(36+64) = sqrt(100) = 10.

Q3. Compute the dot product of p=[3,6,9] and q=[1,0,-1], showing steps.

A: (3*1)+(6*0)+(9*-1) = 3+0-9 = -6.

Q4. What is the difference between len(vector) and np.linalg.norm(vector)?

A: len() counts the number of elements (dimension/count). np.linalg.norm() computes the vector's geometric magnitude (distance from the origin). Two unrelated quantities.

Quiz Q&A; -- Matrices

Q1. Can you multiply a 3×4 matrix by a 4×2 matrix? Dimensions of result? What about 2×3 by 4×5?

A: 3×4 by 4×2: valid (inner dims 4,4 match) -> result 3×2. 2×3 by 4×5: INVALID (inner dims 3 vs 4 don't match).

Q2. Given M=np.array([[10,20,30],[40,50,60]]), get row index 1, all columns.

A: M[1, :] -> [40 50 60]

Q3. Given P=[[1,2],[3,4]] and Q=[[2,0],[1,2]], compute P@Q by hand, showing each entry.

A: Result[0,0]=(1*2)+(2*1)=4. Result[0,1]=(1*0)+(2*2)=4. Result[1,0]=(3*2)+(4*1)=10. Result[1,1]=(3*0)+(4*2)=8. P@Q = [[4,4],[10,8]].

Q4. True or False: A*B and A@B always give the same result for two matrices.

A: False. * multiplies elementwise; @ performs true matrix multiplication (row·column dot products). Fundamentally different operations, generally different results.

Q5. Why must the 'inner dimensions' of two matrices match for multiplication to be valid? Tie it to the dot product.

A: Each result entry is the dot product of one row of A and one column of B. A dot product needs two vectors of EQUAL length. A row's length equals A's number of columns; a column's length equals B's number of rows -- so those two counts must match, or there's nothing valid to pair up.